

Représentation des graphes – Sujet

Exercice d'application

On donne la carte de la région Nouvelle Aquitaine.

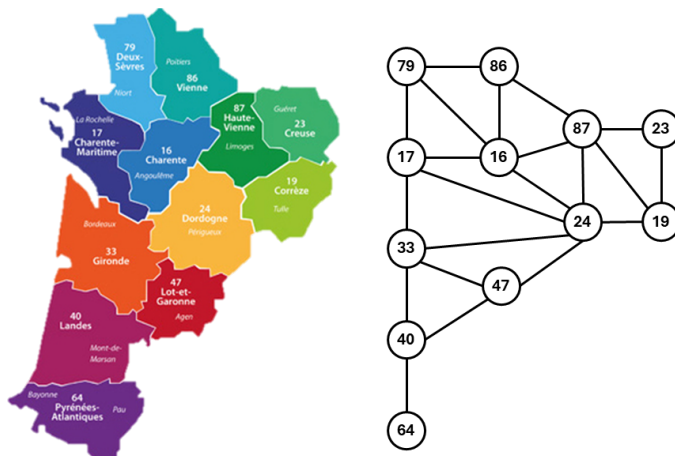


FIGURE 1 – Carte et graphe de la région Nouvelle-Aquitaine

Dictionnaire

On souhaite créer un dictionnaire permettant de faire une correspondance entre le numéro des départements et leur nom.

Question 1 Implémenter une fonction `add_dpt(d:{}, dpt:int, nom:str) -> None` qui ajoute un département au dictionnaire `d`.

Dictionnaire d'adjacence

On souhaite modéliser la carte par un dictionnaire d'adjacence où les sommets sont des numéros de départements et les valeurs sont la liste des départements limitrophes.

Question 2 Donner l'instruction permettant de créer un graphe avec les Pyrénées-Atlantiques (64) et les Landes uniquement (40).

Question 3 Implémenter la fonction `add_dpt_d(G:{}, dpt:int, voisins : [int]) -> None` qui ajoute un département au graphe G.

Question 4 Implémenter la fonction `add_arete_d(G:{}, dpt1:int, dpt2:int) -> None` qui ajoute une arête entre deux départements.

Question 5 Implémenter la fonction `voisins_d(G:{}, dpt:int) -> [int]` qui renvoie la liste des départements voisins de `dpt`.

Question 6 Implémenter la fonction `del_arete_d(G:{}, dpt:int) -> None` qui supprime un département de la région.

Question 7 Implémenter la fonction `degre_d(G:{}, dpt:int) -> int` qui donne le degré d'un sommet.

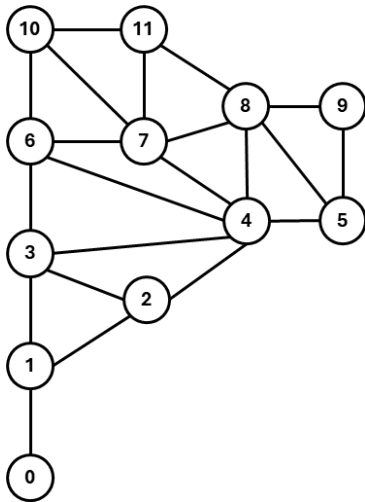


FIGURE 2 – Re-numérotation des sommets.

Question 8 Implémenter la fonction `degre_max_d(G:{}) -> int` qui renvoie le sommet de plus haut degré.

Question 9 Implémenter la fonction `degre_max_d(G:{}, d:{}) -> int` prend en argument le graphe de la région ainsi que le dictionnaire des départements qui renvoie le nom du (ou des) départements ayant le plus de départements limitrophes.

Liste d'adjacence

On souhaite modéliser la carte par une liste d'adjacence où les sommets sont des numéros de départements et les valeurs sont la liste des départements limitrophes.

La numérotation des départements est modifiée conformément à la figure ci-contre.

Question 10 Donner l'instruction permettant de créer un graphe avec les départements 0, 1, 2, 3.

Question 11 Implémenter la fonction `add_dpt_l(G:[], dpt:int, voisins : [int]) -> None` qui ajoute un département au graphe G.

Question 12 Implémenter la fonction `add_arete_l(G:[], dpt1:int, dpt2:int) -> None` qui ajoute une arête entre deux départements.

Question 13 Implémenter la fonction `voisins_l(G:[], dpt:int) -> [int]` qui renvoie la liste des départements voisins de dpt.

Question 14 Implémenter la fonction `del_arete_l(G:[], dpt:int) -> None` qui supprime un département de la région.

Question 15 Implémenter la fonction `degre_d_l(G:[], dpt:int) -> int` qui donne le degré d'un sommet.

Question 16 Implémenter la fonction `degre_max_d(G:[]) -> int` qui renvoie le sommet de plus haut degré.

Matrice d'adjacence

On souhaite modéliser la carte par une matrice d'adjacence. On conserve la numérotation précédente.

Question 17 Donner l'instruction permettant de créer un graphe avec les départements 0, 1, 2, 3.

Question 18 Implémenter la fonction `add_dpt_m(G:[[]], dpt:int, voisins : [int]) -> None` qui ajoute un département au graphe G.

Question 19 Implémenter la fonction `add_arete_m(G:[[]], dpt1:int, dpt2:int) -> None` qui ajoute une arête entre deux départements.

Question 20 Implémenter la fonction `voisins_l(G:[[]], dpt:int) -> [int]` qui renvoie la liste des départements voisins de dpt.

Question 21 Implémenter la fonction `del_arete_m(G:[[]], dpt:int) -> None` qui supprime un département de la région.

Question 22 Implémenter la fonction `degre_d_m(G:[[]], dpt:int) -> int` qui donne le degré d'un sommet.

Question 23 Implémenter la fonction `degre_max_m(G:[[]]) -> int` qui renvoie le sommet de plus haut degré.

Déplacement d'un cavalier sur un échiquier

Un cavalier se déplace, lorsque c'est possible, de 2 cases dans une direction verticale ou horizontale, et de 1 case dans l'autre direction (le trajet dessine une figure en L).

Dans un premier temps, les cases de l'échiquier sont représentées par des tuples : le couple (i, j) désigne la case d'abscisse i et d'ordonnée j . Un échiquier possède 8 colonnes et 8 lignes, donc i et j seront compris entre 0 et 7.

Question 24 Écrire une fonction `estDansEch(i:int, j:int) -> bool` qui renvoie True si (i, j) correspond à une case valide de l'échiquier et False sinon.

Question 25 Écrire une fonction `mvtsPossibles(i:int, j:int) -> list` qui renvoie la liste des cases où le cavalier peut se déplacer à partir de la case (i, j) à l'ordre près.

Question 26 Vérifier que :

- ▶ `mvtsPossibles(0,0)` renvoie $[(1, 2), (2, 1)]$,
- ▶ `mvtsPossibles(3,5)` renvoie bien $[(1, 4), (1, 6), (2, 3), (2, 7), (4, 3), (4, 7), (5, 4), (5, 6)]$,
- ▶ `mvtsPossibles(7,7)` renvoie bien $[(5, 6), (6, 5)]$.

Tous ces résultats sont à l'ordre près.

Question 27 Créer un graphe G sous la forme d'un dictionnaire d'adjacence avec pour sommets les différentes cases de l'échiquier et les arêtes qui correspondent à un mouvement possible du cavalier.

Question 28 Vérifiez que vous avez un graphe avec 64 sommets et 168 arêtes.

Le codage des cases d'échiquier se fait classiquement par une chaîne de caractères comprenant : une lettre minuscule pour l'abscisse (de a à h) et un chiffre pour l'ordonnée (de 1 à 8). La case en bas à gauche de l'échiquier est donc de code 'a1'.

Remarque

`ord(c)` retourne le codage Unicode correspondant au caractère c et `chr(n)` renvoie le caractère dont le codage Unicode est n .

Question 29 Écrire une fonction `codage(i:int, j:int) -> str` qui renvoie le code correspondant à la case (i, j) .

Question 30 Créer un dictionnaire d'adjacence $G2$ comme défini précédemment avec les cases maintenant nommées d'après leur code.

Question 31 Vérifier que, à l'ordre près :

- ▶ `G2['a1']` renvoie $['b3', 'c2']$,
- ▶ `G2['d6']` renvoie bien $['b5', 'b7', 'c4', 'c8', 'e4', 'e8', 'f5', 'f7']$,
- ▶ `G2['h8']` renvoie bien $['f7', 'g6']$.

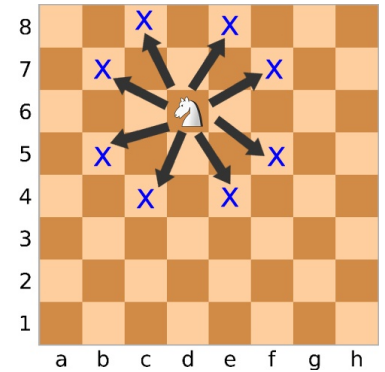


FIGURE 3 – Illustration du mouvement d'un cavalier sur un échiquier

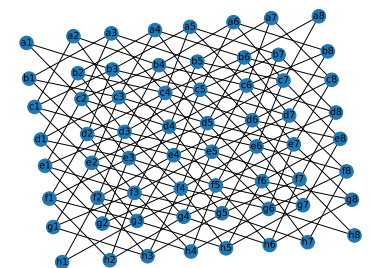


FIGURE 4 – Représentation graphique du graphe $G2$